JOSEPHIAN SCHOOLS

Consolidation and Engineering Review

Exams and Report Cards Module - Page Merge Analysis and Go-Ahead Decision

Prepared for: Deo, Project Owner

Scope: 23 pages, 7,460 lines, Exams and Report Cards module

Method: agent-skills audit with static verification

Status: For decision - no code changed yet

Date: 4 November 2026

Table of Contents

1. Executive Summary....1
2. Background, Scope and Method....2
2.1 Purpose of This Report....2
2.2 Scope and Evidence Base....2
2.3 Agent Skills Applied....2
3. Current State: Strengths to Preserve....2
4. Engineering Audit Findings....3
4.1 Duplicated Logic Across Pages....3
4.2 Configuration Drift and Magic Values....4
4.3 Dead and Unused Code....4
5. Page Consolidation Proposals....4
5.1 Proposal 1: Merge Readiness Check with Marks Deadline Tracker....5
5.2 Proposal 2: Merge Review and Approval with Release Management....5
5.3 Proposal 3: Merge the Parent Portal Report Cluster....5
5.4 Proposal 4: Merge the Cycle Analytics Cluster....6
5.5 Proposal 5: Merge Class Results with Next-term Targets....6
5.6 Pages That Should Not Be Merged....6
5.7 Effect on Navigation....7
6. Route Preservation Strategy....7
7. Implementation Roadmap and Effort....8
8. Risks and Mitigations....8
9. Decision Requested....8
Appendix A: Page Inventory....10

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1. Executive Summary

This report answers two questions posed before the backend-integration phase of the Exams and Report Cards module: does the current implementation follow sound software engineering principles, and which of the twenty-three pages can be combined or merged. The audit covered all twenty-three route pages (7,460 lines of page-level code), the application shell, the shared component library, and the data layer in src/lib. Every finding quoted in this report was verified with exact-pattern static searches against the codebase; no code has been modified yet.

The verdict on engineering principles is positive at the foundation and mixed at the page layer. The strengths are real and worth protecting: a single centralized demo configuration file controls all demonstration identifiers (cy1, the Form 3A class identifier, and st1), one navigation registry drives the sidebar, breadcrumbs, command palette and home page, and the heavy domain mathematics (grades, averages, cycle statistics) already lives in shared modules. At the page layer, however, the audit confirmed fifteen distinct duplication patterns. The most significant are: route-parameter boilerplate repeated in fourteen files, the parent-portal child selector copy-pasted in four pages, CSV export logic in four pages, two differently-structured hash functions sharing one name, and completion-percentage arithmetic recomputed in at least six places. None of these is a functional defect today, but each is a maintenance liability and several will be touched again during backend integration, which multiplies the cost of leaving them in place.

Five consolidation opportunities were identified. If all five are approved, sidebar navigation is simplified from twenty-three entries to sixteen, while every one of the twenty-three existing URLs continues to resolve exactly as today. The mechanism is a host-component pattern: each merged feature becomes one shared page host with tabs, and the existing route files become thin wrappers that render the host with the appropriate tab preselected. No redirects, no new route segments, and no change to direct-URL or refresh behaviour. The consolidation also enables an estimated net removal of 700 to 1,000 lines (roughly nine to thirteen percent of the page layer) and eliminates an entire class of divergence bugs, such as two submission-status classifiers that already exist under different names with the same thresholds.

Implementation is planned in three independently verifiable waves: shared-logic extraction first (no behaviour change), then the three highest-value page merges, then the remaining merges and two cross-page component extractions. Section 9 lists the exact decisions requested. Nothing in this plan changes demo behaviour, business logic, permissions, or the backend-integration approach.

2. Background, Scope and Method

2.1 Purpose of This Report

The redesigned Exams and Report Cards module is functionally complete in its demonstration implementation: twenty-three pages across four navigation groups (Exams, Report cards, Classes, Parent portal), a collapsible sidebar, a centralized design-token system in the Josephian burgundy palette, and a configuration layer that keeps all demonstration identifiers in one file. Before connecting real backend data, the project owner asked whether the modules follow best software and engineering practices, and which pages could be combined or merged. Per instruction, this report presents the analysis and stops at the decision point; implementation begins only after a go-ahead.

2.2 Scope and Evidence Base

The audit examined the full page layer (twenty-three page files, 7,460 lines in total, listed with routes and line counts in Appendix A), the shell components (app-sidebar, app-header, app-shell, command-palette), the shared component library (page-header, section-card, stat-card, status-badge, grade-pill, empty-state), the lib layer (demo-config, demo-data, store, nav, grading, ui-store, types, utils), and the Prisma schema. The method was static analysis: exact-pattern searches with ripgrep to count duplication sites, line-level reading of each page to establish data dependencies and unique functionality, and cross-referencing of imports to build a component-usage matrix. Quantitative claims in this report are reproducible searches, for example the fourteen files containing the identical cycle-resolution expression, or the four portal pages containing the same child-selector component.

2.3 Agent Skills Applied

The audit and the proposals in this report were produced with the installed agent-skills library (addyosmani/agent-skills), applying its workflows as follows. The using-agent-skills meta-skill governed discovery and its operating behaviours: assumptions surfaced before analysis, scope discipline, and verification with evidence rather than assumption. The code-review-and-quality skill supplied the five-axis review frame (correctness, readability, architecture, security, performance) used in Section 4. The code-simplification skill supplied the duplication-detection patterns and two governing rules that shape the plan: understand before removing (Chesterton's Fence) and preserve behaviour exactly during the extraction wave. The frontend-ui-engineering skill supplied the composition-over-configuration and purpose-driven-layout principles that define the do-not-merge list in Section 5.6. The planning-and-task-breakdown and incremental-implementation skills shaped the wave plan in Section 7, and the deprecation-and-migration skill informed the route-preservation strategy in Section 6, since the original redesign brief requires all existing routes to keep working. The earlier-

installed plugin87/ux-ui-agent-skills remains available as a UX reference set and was not needed for this analysis.

3. Current State: Strengths to Preserve

Before listing findings, it is worth recording what the implementation already gets right, because the consolidation plan depends on these structures and must not damage them.

Centralized demonstration configuration. One file, src/lib/demo-config.ts, defines DEMO_IDS (report cycle cy1, class 01a1a064-... for Form 3A, student st1), the SCHOOL branding record, the resolveHref template resolver, and a routes builder for every page. Every route in the application is built from dynamic segments, so replacing the demonstration identifiers with real backend identifiers is a one-file change. This is exactly the structure the original redesign brief required, and it is in place.

A single navigation registry. src/lib/nav.ts defines all twenty-three navigation entries as templates containing {cycleId}, {classId} and {studentId} placeholders. The sidebar, breadcrumbs, command palette, active-state highlighting (regex matching with longest-template-wins) and the home-page module map all derive from this one registry. Any consolidation of navigation is therefore a change to one data structure, not to many components.

A shared component library and central domain mathematics. Seven shared components (page-header with document titles, section-card, stat-card, status-badge with a centralized status vocabulary, grade-pill, empty-state) are used across the pages, and grading.ts centralizes grade lookup, tone classes, averaging, pass-rate and formatting. The derived statistics (classResults, cycleStatsFor, subjectAveragesFor, commentProgress, departmentStats) live in demo-data.ts rather than in pages. This is the architecture the code-review-and-quality skill's architecture axis asks for; the findings below are therefore about the page layer, not the foundation.

Deep-linkable page design. All pages are client components that read identifiers from the URL and fall back to DEMO_IDS when parameters are absent, so direct URL access and refresh work for any cycle, class or student identifier, which was another hard requirement of the redesign brief.

4. Engineering Audit Findings

4.1 Duplicated Logic Across Pages

The table below lists the fifteen verified duplication patterns. The count column is the number of independent copies found by exact search. The risk column records why the duplication matters in practice rather than in theory.

Table 1: Verified duplication patterns in the page layer

#	Pattern	Where (independent copies)	Risk
1	Cycle-context boilerplate: cycleById(params.cycleId ?? "") ?? currentCycle	14 pages	A change to cycle resolution during backend integration must be edited fourteen times; one missed page silently uses the fallback cycle
2	Class-context variant of the same expression	5 pages	Same liability at class level
3	Parent-portal child bundle: children list, firstName, initialsOf and a roughly 45-line ChildChips component	4 portal pages	About 180 duplicated lines; the exported demoParent record that should feed it is unused
4	CSV export via Blob and synthetic anchor click	exams, batches, dashboard, results (4)	Encoding or quoting fixes must be repeated four times
5	window.print PDF helper with near-identical toast copy	report-card preview, portal report card (2)	Print-layout plumbing duplicated across staff and parent views of the same document
6	hashStr defined twice with two different algorithms (FNV-1a in demo-data, base-31 variant in portal progress)	2 definitions	Name collision invites wrong-import bugs; pseudo-random previous-term deltas are not reproducible across pages
7	Completion-percentage arithmetic recomputed locally	readiness, batches, departments, deadlines (two forms), dashboard, marks (6 or more sites)	Threshold drift already visible between sites; readiness per-class aggregation and batch validation re-implement the same checks
8	Submission-status classifier (done and total mapped to complete, partial, overdue)	deadlines statusOf vs departments submissionStatus (2, plus a third label variant in batches)	The same state can be labelled differently on two pages
9	Grade-distribution counting loop	demo-data internals plus a local re-implementation in class results (2 layers)	Same mathematics maintained in two places
10	Lowest-subject detection chain (filter, ascending sort, first)	results, targets, departments, dashboard (3 to 4 sites)	Identical selection logic copy-pasted
11	Booking-conflict predicate and slotStart time helper	staff parents-day, portal parents-day (2 each)	A business rule duplicated across two audiences; drift risk when the rule changes
12	Date formatting fmtLongDate with a private MONTHS array	report-card, release, portal parents-day (3)	Three copies of a formatter
13	Status-tab filter pattern (tabs, filter expression, counts)	exams, deadlines, review (3)	Interaction pattern re-implemented rather than shared

#	Pattern	Where (independent copies)	Risk
14	Hardcoded approver and actor name	report-cycles, review, release, demo-data (4)	Actor identity scattered; a real staff change touches four files
15	Hardcoded bursary contact details on the on-hold page, with an email that differs from the SCHOOL record	portal on-hold (1, versus demo-config)	Configuration drift: a real contact change would miss this page

Read against the five axes of the code-review-and-quality skill, the findings concentrate on the architecture and readability axes. Correctness shows no functional defect: the demonstration behaves as designed on all routes. The duplication is concentrated exactly in the code that backend integration will later touch most: identifier resolution, completion thresholds, status classification, formatting, and contact configuration. Extracting these before the backend phase is materially cheaper than extracting them during it.

4.2 Configuration Drift and Magic Values

Four values that should be named configuration are currently literals. The bursary contact block on the portal on-hold page hardcodes both a telephone number and the address accounts@josephian.ac.tz, while the SCHOOL record in demo-config.ts carries info@josephian.ac.tz; the two are inconsistent today. The management dashboard compares the current cycle average against the literal 62.4 as the previous-term baseline without a name or comment explaining its provenance. The report-cycles page carries a local DEMO_TODAY constant set to 11 March 2026, and the review page embeds hardcoded audit timestamps. Each is harmless in a demonstration and each becomes a defect vector once real data flows; all four belong in the configuration layer with the other demo constants.

4.3 Dead and Unused Code

The audit found five unused artefacts. The Prisma schema still contains the scaffold User and Post models and lib/db.ts is imported nowhere in the application, which is expected before backend integration and is flagged here only so it is consciously retained or replaced during that phase. The MarkEntry interface and the rankOf helper are exported but unused by any page. The demoParent record is exported but unused, while four portal pages hand-build the same parent-child mapping it should supply. Three cycle-scoped pages (review, batches, release) declare a studentId in their route-parameter types that the routes never carry, residue of the template they were built from. Together with the duplication in Table 1, the page layer can lose an estimated 700 to 1,000 lines without any behaviour change.

5. Page Consolidation Proposals

Five merge candidates survived the evidence review. The selection criteria were: the pages share one data domain and one primary audience; their union forms a single coherent workflow moment; the merged host does not exceed a comfortable page complexity; and the existing routes can be preserved with the thin-wrapper pattern described in Section 6. Each proposal below states the evidence, the merged shape, and what remains unique to each side. Approvals are requested per proposal, not as a package.

5.1 Proposal 1: Merge Readiness Check with Marks Deadline Tracker

Pages: readiness (353 lines) and deadlines (251 lines), both scoped to a report cycle. Evidence: the two pages monitor the identical inputs, namely the marksDeadlines sheets, commentProgress and the approval state. The readiness page's blocking-issues list re-renders the deadlines page's overdue rows; its per-class eligibility table repeats the deadline tracker's per-class progress bars; and its per-class aggregation is the same computation the batch page re-implements as validationFor. The readiness page's Recalculate button is currently a toast-only no-op, so the gate is a view over the same data rather than a separate computation.

Merged shape: one host, Cycle readiness and deadlines, with two tabs. The Deadline tracker tab keeps the class-by-subject sheet table with its status tabs, filters and reminder action. The Readiness gate tab keeps the readiness donut, the blocking-issues list and the per-class eligibility table. The staff workflow becomes one continuous story: track submissions, open the gate, proceed to batch generation. Strength: strong. Routes preserved: /academic/report-cycles/{cycleId}/readiness opens the host on the gate tab; the deadlines route opens it on the tracker tab.

5.2 Proposal 2: Merge Review and Approval with Release Management

Pages: review (367 lines) and release (385 lines), both cycle-scoped admin gates. Evidence: the two pages share an identical component signature, four summary stat cards, a per-class table, AlertDialog-confirmed actions and status tabs, built on the same per-class state machine pattern. The release page re-displays the approval column that the review page owns, and the two stages are sequential in the real workflow (a class is approved, then released) by the same audience, the school administrator. Both maintain a history timeline (audit trail and release history respectively).

Merged shape: one host, Approvals and release, presenting the pipeline Draft, Submitted, Approved, Released with per-class actions, plus two tabs for the detail views: Approvals (with the audit-trail dialog) and Release and holds (with hold reasons and the release

timeline). Strength: strong. Routes preserved: the review route opens the approvals tab; the release route opens the release tab.

5.3 Proposal 3: Merge the Parent Portal Report Cluster

Pages: portal report-cards (277 lines), portal report-cards/on-hold (263 lines) and portal progress (298 lines). Evidence: this is the highest-duplication cluster in the module. The child-selector bundle (children list, firstName, initialsOf and the ChildChips component) is copy-pasted in all four portal pages, three of which this merge absorbs. The on-hold page is not an independent feature but a state of the report-card flow: the report-card page already renders a hold-warning strip that links to it, so a parent is currently sent to a second page to learn why the first page's document is unavailable. The progress page re-renders subject remarks and target data that the card page and the class results page already compute, via targetFor and subjectRemark.

Merged shape: one parent hub, My children's reports, with the existing child chips and two tabs: Report card and Progress and targets. When the selected child's card is held, the Report card tab renders the hold panel in place, with the reason, the three-step resolution guidance and the bursary contacts, removing a navigation hop in the most sensitive parent journey. Strength: strongest duplication payoff of the five. Routes preserved: /portal/report-cards opens the hub on the card tab; /portal/report-cards/on-hold is kept as a thin route that opens the hub with the hold state forced, so deep links from messages or letters keep working; /portal/progress opens the progress tab.

5.4 Proposal 4: Merge the Cycle Analytics Cluster

Pages: dashboard (307 lines), departments (224 lines) and public-summary (236 lines), all cycle-scoped. Evidence: all three pages are views over the same cycleStatsFor statistics object. The departments page's subject-averages bar duplicates the dashboard's chart; the public-summary page re-renders the dashboard's grade-distribution and class-average charts as its preview; and the dashboard, in turn, is the only consumer of the deltaLabel comparison against the previous-term baseline. The departments page has genuinely unique content, the HOD teacher-submission tables, which is the only reason this proposal is rated medium-strong rather than strong.

Merged shape: one host, Cycle analytics, with three tabs: Overview (the executive dashboard), Departments (HOD view), and Public summary (publication toggles with the live preview). Strength: medium-strong, because the audiences differ even though the data is identical. A lighter alternative is approved if preferred: merge only dashboard and public-summary and leave departments standalone, which yields a navigation of seventeen entries instead of sixteen. Routes preserved under either variant.

5.5 Proposal 5: Merge Class Results with Next-term Targets

Pages: results (286 lines) and targets (286 lines), both scoped to a class within a cycle. Evidence: both are views over classResults, subjectAveragesFor and the default grading scheme. The targets page duplicates the results page's key performance figures and its lowest-subject detection chain; functionally it is the results matrix plus two editable columns (target average and focus area) and a recommendation generator.

Merged shape: one host, Results and targets, with two tabs: Results analysis (order-of-merit matrix, charts, export) and Next-term targets (editable goal table). Strength: medium. Routes preserved: the results route opens the analysis tab; the targets route opens the targets tab.

5.6 Pages That Should Not Be Merged

The frontend-ui-engineering skill's purpose-driven-layout principle cuts both ways: pages whose jobs genuinely differ should stay separate even when they share data. The following were considered and are recommended against merging. Exams, Grading schemes and Report cycles are three distinct configuration domains with different interaction models. Marks entry is a keyboard-driven spreadsheet with cell navigation and dirty-state tracking, the module's most specialised interaction, and it must remain its own page. Comments is a two-pane editor of a different nature from any neighbouring page. The staff report-card preview owns a bespoke A4 print layout; rather than merging it with anything, the plan extracts its card renderer so the parent portal and the staff preview share one ReportCardSheet component, which alone removes the duplicated print plumbing recorded in Table 1. The staff parents-day scheduler and the parent booking page serve opposite sides of the same appointment data with deliberately different interfaces; the plan extracts the shared booking engine (slot helpers, conflict predicate, appointment list) rather than the interface. Class subjects is class-level configuration with a page-local allocation model. Verify is intentionally outside the application shell as a public page and should stay exactly as it is.

5.7 Effect on Navigation

Approving all five proposals, including the full analytics merge, reduces the sidebar from twenty-three entries to sixteen while all twenty-three routes remain reachable. The table below shows the resulting structure. The home-page module map and the command palette regenerate automatically because both derive from the single navigation registry.

Table 2: Navigation before and after full consolidation

Group	Today	After	Change
Exams	2 entries	2 entries	No change

Group	Today	After	Change
Report cards	13 entries	9 entries	Readiness and deadlines become one; review and release become one; dashboard, departments and public summary become one
Classes	3 entries	2 entries	Results and targets become one; class subjects unchanged
Parent portal	5 entries	3 entries	Report card, on-hold and progress become one hub; booking and verify unchanged
Total	23 entries	16 entries	All 23 URLs still resolve; 7 entries consolidated into 5 hosts

6. Route Preservation Strategy

The original redesign brief makes route preservation a hard requirement: every existing route must keep working, directly by URL and on refresh, with no invented routes and no change of route pattern. The consolidation respects this with a host-plus-thin-wrapper pattern instead of redirects.

Each merged feature becomes a single host component, for example CycleReadinessHost, accepting an initialTab property. The existing page.tsx files remain in place at their existing paths and shrink to thin wrappers of roughly ten to fifteen lines that render the host with the appropriate tab preselected. Nothing moves in the URL space: no redirect, no new segment, no client-side rewrite, so deep links, browser history and refresh behave exactly as today. The current cycle, class and student context continues to come from the URL parameters with the existing DEMO_IDS fallback.

Two secondary behaviours are worth noting. First, the active tab can optionally be reflected in a query parameter (for example ?tab=gate) so that a staff member can bookmark or share not just a page but a view of it; this touches no route definitions. Second, the sidebar will show one entry per host, pointing at the primary route of that host. Breadcrumbs and command-palette entries are recomputed from the navigation registry and therefore update themselves. If a future requirement needs a nav entry per tab, the registry supports it without further refactoring.

7. Implementation Roadmap and Effort

The work is staged in three waves so that each step is independently verifiable and reversible. Wave 1 performs only behaviour-preserving extraction, which the code-simplification skill treats as the safest class of refactoring. Waves 2 and 3 perform the approved page consolidations on top of the extracted foundations, which makes each merge mechanically simpler than it would be today.

Table 3: Implementation waves

Wave	Content	Verification gate
Wave 1	Extraction only, no behaviour change: useCycleContext and useClassContext hooks (removes the fourteen-site boilerplate); lib/format.ts (fmtLongDate, fmtStamp, slotStart); lib/csv.ts (one exportCsv); lib/status.ts (one submission classifier); shared child-chips component and useParentChildren for the portal; lib/booking.ts (conflict predicate); named constants for the approver, bursary contacts, baseline average and demo date; removal of dead code listed in Section 4.3	Build passes; all 23 routes loaded and checked; no console errors; visual spot check of one page per group
Wave 2	The three strongest merges: Proposal 1 (readiness and deadlines), Proposal 2 (review and release), Proposal 3 (parent portal hub)	Build passes; all 23 routes plus tab deep-links checked; portal flows walked end to end
Wave 3	Proposals 4 and 5; ReportCardSheet extraction shared by staff preview and portal; booking-engine adoption by both parents-day pages	Build passes; all 23 routes checked; print preview checked on both report-card surfaces

Effort is estimated in working sessions rather than hours: Wave 1 is one session, and Waves 2 and 3 are approximately one session each. The net effect on the codebase is an estimated reduction of 700 to 1,000 lines in the page layer, and a reduction of every pattern in Table 1 from multiple independent copies to one named definition. The demonstration identifiers, the design tokens, the permissions model and the backend-integration plan are untouched throughout.

8. Risks and Mitigations

The plan is deliberately conservative, and each risk below has a concrete mitigation that is part of the wave definition rather than an aspiration.

Table 4: Risks and mitigations

Risk	Mitigation
A regression in one of the 23 routes during consolidation	Thin wrappers keep every route file in place; each wave ends with the same 23-URL checklist; any failed check blocks the wave
Hidden behaviour drift during extraction	Wave 1 is governed by the preserve-behaviour rule: refactor commits separate from any other change, thresholds and labels byte-identical, tests or build gate after each extraction
Demo-identifier coupling breaks when the backend arrives	demo-config.ts remains the single source; the new hooks resolve identifiers through it, so the backend swap still touches one file
Parent confusion when the on-hold page becomes a tab state	The /portal/report-cards/on-hold route is preserved verbatim and opens the hub with the hold panel forced; existing links and letters keep working
Scope creep into feature changes	This plan contains no feature work; new functionality is explicitly out of scope until the consolidation is verified
Analytics merge dilutes an audience-specific view	Proposal 4 offers a reduced variant (dashboard plus public summary only, departments standalone) if the HOD view is better kept isolated

9. Decision Requested

The following decisions are requested. Each is independent; a go-ahead can cover any subset.

1. Approve Wave 1, the shared-logic extraction. Recommended regardless of the merge decisions, because it removes the duplication that backend integration would otherwise multiply. No behaviour changes.
2. Approve or reject Proposal 1: merge Readiness check and Marks deadline tracker into one host with two tabs.
3. Approve or reject Proposal 2: merge Review and approval and Release management into one pipeline host.
4. Approve or reject Proposal 3: merge the parent portal report cards, on-hold and progress pages into one parent hub, with the on-hold route preserved.
5. Approve or reject Proposal 4: merge the analytics cluster into one host with Overview, Departments and Public summary tabs, or its lighter variant without Departments.
6. Approve or reject Proposal 5: merge Class results and Next-term targets into one host.
7. Confirm the navigation consolidation target: sixteen entries with all five proposals, or seventeen with departments kept standalone.
8. Confirm the Wave 3 extractions: ReportCardSheet shared renderer and the shared booking engine.

On a go-ahead, implementation starts with Wave 1 and each wave closes with the verification gate in Table 3 before the next begins. If any proposal is rejected, the affected pages remain exactly as they are today; the extraction wave still removes their duplicated plumbing, and the sidebar keeps the current twenty-three entries.

Appendix A: Page Inventory

The inventory lists every page audited, its route as it exists today, its page-layer line count, and its principal data dependencies. Line counts are from the current working tree (4 November 2026).

Table 5: The twenty-three pages of the Exams and Report Cards module

#	Page	Route	LOC	Principal data dependencies
1	Exams	/academic/exams	379	exams, subjectById, teacherById; status tabs; CSV export

#	Page	Route	LOC	Principal data dependencies
2	Grading schemes	/academic/grading-schemes	326	gradingSchemes, gradeFor; band visualiser and simulator
3	Report cycles	/academic/report-cycles	401	cycles, currentCycle, parentsDayEvent; create-cycle dialog
4	Readiness check	/academic/report-cycles/{cycleId}/readiness	353	marksDeadlines, commentProgress, approvalFor, cycleStatsFor
5	Marks deadline tracker	/academic/report-cycles/{cycleId}/deadlines	251	marksDeadlines, classById, subjectById
6	Marks entry	/academic/report-cycles/{cycleId}/classes/{classId}/marks	375	getMarks, studentsInClass, defaultScheme; keyboard grid
7	Comments	/academic/report-cycles/{cycleId}/classes/{classId}/comments	321	commentFor, commentTemplates, studentsInClass
8	Report card preview	/academic/report-cycles/{cycleId}/students/{studentId}/report-card	289	studentResultOf, studentAttendance, commentFor, verificationRecords; print layout
9	Review and approval	/academic/report-cycles/{cycleId}/review	367	approvals, approvalFor; audit-trail state machine
10	Batch generation	/academic/report-cycles/{cycleId}/batches	516	batchRuns, marksDeadlines, commentProgress, approvalFor; wizard
11	Release management	/academic/report-cycles/{cycleId}/release	385	releaseFor, approvalFor; hold reasons and history
12	HOD department summary	/academic/report-cycles/{cycleId}/departments	224	departmentStats, cycleStatsFor; bar chart
13	Management dashboard	/academic/report-cycles/{cycleId}/dashboard	307	cycleStatsFor, marksDeadlines, defaultScheme; four charts
14	Public summary	/academic/report-cycles/{cycleId}/public-summary	236	cycleStatsFor; publication toggles and preview
15	Parents' day schedule (staff)	/academic/report-cycles/{cycleId}/parents-day	370	bookingSlots, parentsDayEvent; conflict-checked booking
16	Class subjects	/academic/classes/{classId}/subjects	345	subjectTeacherOf, teachers, subjects; allocation editor
17	Class results	/academic/report-cycles/{cycleId}/classes/{classId}/results	286	classResults, subjectAveragesFor, defaultScheme; matrix and charts
18	Next-term targets	/academic/report-cycles/{cycleId}/classes/{classId}/targets	286	targetFor, classResults, subjectAveragesFor
19	Report card (parent)	/portal/report-cards	277	studentResultOf, studentAttendance, commentFor, releaseFor; print
20	Card on hold (parent)	/portal/report-cards/on-hold	263	releaseFor; hold reason, resolution steps, bursary contacts
21	Progress and targets (parent)	/portal/progress	298	studentResultOf, targetFor, subjectRemark; previous-term deltas
22	Book parents' day (parent)	/portal/parents-day	373	bookingSlots, parentsDayEvent; slot grid booking
23	Verify a report card (public)	/verify	232	verifyCode, verificationRecords, classResults; standalone layout